

AICAS 2026 Qwen3-VL-2B 推理优化项目技术报告

队伍名：ict-Jack

参与者：何逸天，陈泽森，余大川

指导教师：刁博宇

所属机构：中国科学院计算技术研究所；中国科学院大学

1. 引言

随着视觉语言模型（Visual Language Model, VLM）技术的快速发展，多模态大模型在图文问答、视觉理解和智能助手等任务中展现出优异性能。然而，这类模型通常参数规模较大、推理链路较长，并且同时包含视觉编码、跨模态特征注入和语言生成等多个阶段，在固定硬件和在线推理场景下往往面临较高的首 token 延迟与持续生成开销。对于单卡部署、边缘推理以及受限评测环境而言，如何在保证模型语义与输出质量不变的前提下有效压缩推理时延、提升吞吐率，已成为视觉语言模型落地中的关键问题。

在本次推理优化项目中，我们采用了一种综合系统优化方案。我们首先接管官方接口，在输入阶段提取结构信息，然后根据请求类型选择不同路线，视觉侧通过分辨率预算、特征缓存和图回放降低首 Token 成本，prefill 阶段通过专用路径、结构模板和图回放减少启动开销，decode 阶段通过静态缓存切换、whitebox token-block、持久图和 tail 策略提高 128 token 吞吐，算子层面通过 QKV 融合、MLP gate/up 融合、W8A16 RowTriton、融合 attention 和 top-1 专用化降低单步计算成本。

本文其余部分将详细介绍本项目的方法与实现。并进一步展开优化算法的关键设计，我们希望本项目的实践能够为受规则约束的视觉语言模型高效部署提供有价值的工程参考。

2. 优化技术

2.1. 视觉链路优化

视觉语言模型的首 Token 时间很大一部分来自视觉侧处理，包括图像预处理、视觉 encoder、视觉特征插入和多模态位置编码。我们对视觉链路的优化主要围绕少算、复用、稳定形状三个方向展开。

首先是**请求级视觉预算控制**。并不是所有问题都需要同等高分辨率。对于 OCR、数字识别和计数类问题，图像细节非常重要，因此需要保留更多视觉信息；对于颜色、属性或一般场景理解问题，过高分辨率未必带来明显收益，反而会增加视觉 token 数和 prefill 成本。因此系统会根据问题类型和图片尺寸选择不同视觉预算。性能测量路径更倾向于降低视觉成本，准确率路径则更倾向于保留视觉细节。

其次是**视觉特征缓存**。评测过程中，同一张图片往往会被连续用于首 Token 测试和吞吐测试。如果每次都重新跑视觉 encoder，就会产生重复成本。我们会为图片视觉特征建立缓存，在输入内容和处理策略一致时复用已有视觉输出。

第三是**视觉图回放**。对于常见图片尺寸和视觉网格，系统提前准备静态执行图。请求命中相同形状时，只需要把新的图片输入放入预分配缓冲区，然后直接回放视觉计算图。这样可以减少动态图调度、临时内存分配和大量小 kernel 启动开销。

最后，我们还对**视觉侧局部热点操作做了快路径处理**，例如视觉位置编码和 patch 合并。这类优化主要是减少中间张量和通用框架调用。真实推理场景中，所有输入侧开销都会直接计入首 Token 时间，因此这些细节能够叠加成稳定收益。

2.2. TTFT 优化

Prefill 是从完整图文输入到第一个 token 的关键阶段。标准流程会执行完整语言模型前向，并准备后续生成所需的缓存。对于只生成 1 个 token 的 TTFT 请求而言，部分通用准备并不是必要的，因此我们为首 Token 设计了更轻量的 prefill 路线。

如前文第1节所讲，系统尽量复用输入阶段已经获得的结构信息和视觉特征，直接完成必要的语言模型计算，减少通用生成框架中的多余状态管理。由于首 Token 请求只需要得到第一个 token，不需要为长序列 decode 做完整准备，所以可以减少部分缓存和调度开销。

同时，**结构模板缓存**也服务于这一阶段。多模态输入中有许多结构张量，例如视觉位置掩码、图片 token 区间和位置编码索引。这些张量不包含新的语义信息，但每次构造都会消耗时间。系统会缓存合法且常见的结构模板，下次遇到相同结构时直接复用。

2.3. Decode 与 128 Token 吞吐优化

标准生成流程通常是逐 token 运行：每生成一个 token，都要调用一次模型、更新 KV Cache、计算注意力和 MLP，再计算下一个 token。这种方式在通用框架中很灵活，但在固定 batch 为 1、贪心解码、目标长度明确的场景下会带来大量重复调度成本。

我们的主要改造是**将逐 token decode 变成块级 decode**。系统先完成 prefill，并得到初始 KV Cache；随后将动态缓存切换到固定长度的静态缓存；再根据当前 prompt 长度和目标生成长度选择合适的静态 bucket；最后进入 whitebox token-block 路径，一次生成一段 token，而不是每个 token 都回到通用生成框架。

静态缓存切换是这一路径的关键桥梁。动态缓存灵活但形状不稳定，不利于提前生成执行图。静态缓存长度和布局固定，更适合图回放和静态模块执行。只要 bucket 能容纳当前 prompt 和生成长度，系统就会尝试把已有 KV Cache 转入静态缓存，然后进入专用 decode 路径。

Whitebox token-block 是吞吐提升的核心来源。它利用预生成的静态执行模块，将多个 decode step 合并到同一执行流程中，减少 Python 循环、状态绑定、缓存检查和小 kernel 调用。对于 128 token 这种固定长度请求，多个 token 共用一次更大的执行单元，单 token 平摊开销明显降低。

在此基础上，系统还会尽量使用**持久序列图**。持久图将输入 token 缓冲区、位置状态、静态缓存和输出缓冲区组织成可复用状态，请求命中时直接回放较长的一段生成过程。这样可以进一步减少 block

之间的重复准备成本。

此外，当模型已经进入结束、空白、格式化或稳定 tail 状态时，继续完整 decode 的收益很低。系统会通过验证机制确认 tail 是否稳定，确认后用更低成本完成剩余部分。若模型已经自然生成结束符，就保留第一个结束符，并用 pad token 填充剩余位置，避免继续无意义 decode。

2.4. 算子融合与量化加速

在前文基础上，我们进一步优化语言模型中的热点算子。

首先是 **QKV 融合**。注意力层中 Query、Key、Value 通常来自同一个输入，但标准实现可能进行多次独立线性计算。我们将这三部分计算组织为更连续的执行方式，一次得到三类结果后再切分。这样可以减少线性层调用次数、中间张量和调度成本。

其次是 **MLP 中 gate/up 融合**。Qwen 类模型的 MLP 通常包含 gate 分支和 up 分支，两者输入相同但需要分别计算。我们将两部分权重组织在一起，一次计算得到 gate 和 up，再完成激活、乘法和后续投影。Decode 阶段每一层每个 token 都会经过 MLP，因此这类融合能够在连续生成中持续积累收益。

第三是 **W8A16 RowTriton gate/up**。Decode 场景通常是单 token 或小批量矩阵向量乘，传统高精度通用 GEMM 不一定最适合。我们使用 8 位权重和较高精度激活的混合方式，并配合更适合行主序访问的专用 kernel，加速 MLP 的 gate/up 部分。实验表明，这种方法不是全层启用就最好，而是需要选择合适的层段和调用位置。最终策略对准确率路径较保守，对 128 token 性能路径更积极。

第四是**融合 decode attention**。Decode attention 包含注意力分数计算、softmax、value 聚合和 head 合并等步骤。将部分步骤融合，可以减少中间结果和内存访问。实验中，不同融合模式的稳定性并不相同，因此默认保留收益更稳的模式，而不是盲目启用所有候选。

最后是**输出层 top-1 专用化**。评测使用贪心解码，每一步只需要得分最高的 token，并不需要复杂采样逻辑。针对这一点，系统使用更贴合 top-1 选择的输出路径，减少完整 logits 保存和后处理开销。

3. 结果

上述方法在 TTFT 和吞吐两个维度都取得了显著提升，在 random150 性能样本上的观测结果为：平均 TTFT 约 16.62 毫秒，平均吞吐约 1163 token/s。这些结果表明，当前方案能够在不修改模型权重、不过度依赖单一算子技巧的前提下，显著改善视觉语言模型在受限评测环境中的推理效率，并将原始模板中的通用生成路径有效收束为更适合单请求部署的专用运行时。